Sardar Vallabhbhai National Institute of Technology
Surat - 395007
Department of Artificial Intelligence
Probabilistic Graphical Models (AI453)

[Lab Practical - 7]

Problem Statement: Design and Implementation of **Hidden Markov Models** — the **Forward-Backward and Viterbi Algorithms** — for inferring a hidden state sequence from observations.

INTRODUCTION

Every network studied so far has been static: one set of variables, observed once. A great many problems are not of that kind. Speech, handwriting, gene sequences, load profiles and part-of-speech tags all arrive as sequences, in which the quantity of interest is hidden and only a noisy signal of it is visible.

A hidden Markov model is the simplest graphical model for this situation. A chain of hidden states evolves by a Markov transition rule, and each state emits one observation. The graph is a chain, so it has no loops, and every inference question can be answered exactly and in time linear in the length of the sequence.

Key Insight: There are two different questions one can ask about the hidden states, and they have different answers. *What is the most likely state at position t?* is answered by the forward-backward algorithm, independently at each position. *What is the most likely sequence of states?* is answered by the Viterbi algorithm. Stringing together the most likely state at each position does not generally produce the most likely sequence, and may even produce a sequence the model considers impossible. The two algorithms differ in exactly one operation — a sum becomes a maximum — and that single change is the whole of the difference.

THE MODEL

The occasionally dishonest casino of Durbin, Eddy, Krogh and Mitchison (1998), the standard worked example for hidden Markov models. A dealer rolls a die repeatedly. Most of the time the die is fair, but occasionally it is swapped for a loaded one and swapped back later. Only the sequence of rolls is visible; which die was in use is hidden.

The model is fully specified below. There is no dataset to download and nothing is carried over from any earlier practical.

Hidden states: Fair (F) and Loaded (L). **Observations:** the roll, an integer from 1 to 6.

Initial distribution: P(F) = 0.5, P(L) = 0.5.

Transition probabilities:

From \ to	Fair	Loaded
Fair	0.95	0.05
Loaded	0.10	0.90

Emission probabilities:

Roll	1	2	3	4	5	6
Fair	1/6	1/6	1/6	1/6	1/6	1/6
Loaded	0.10	0.10	0.10	0.10	0.10	0.50

A fixed observation sequence of 30 rolls, to be used wherever a specific sequence is called for, so that every answer is checkable:

```
6 1 5 2 4 1 3 6 4 2 1 2 6 4 1 1 6 2 5 6 5 6 2 6 6 6 6 6 5 6
```

Constraints: Python 3 standard library only. `hmmlearn` and `pgmpy` implement all of these algorithms directly and are not permitted. Index positions from 1 to T in your write-up and from 0 in your code, and say which you are using wherever a position is quoted.

Problem Statement 1: The Model and the Generative Process

Represent the model and generate data from it, so that later answers can be scored against a hidden path that is actually known.

Tasks:

a) Represent the initial distribution, the transition matrix and the emission matrix, and verify that every row of each sums to 1

b) Draw the model as a graphical model for T = 4: four hidden nodes in a chain, each with one observation hanging off it. State which conditional independences the chain asserts, in particular what the state at position t separates

c) Implement a generator that returns both the observation sequence and the hidden path that produced it. Generate 300 rolls and report the proportion of time the loaded die was in use, alongside the stationary probability of the loaded state implied by the transition matrix

d) Report how many sixes appear in the supplied 30-roll sequence and how many a fair die would be expected to produce. State whether that difference alone is sufficient to locate the loaded stretch

Problem Statement 2: The Forward Algorithm

Compute the probability of an observation sequence under the model. Summing over all state paths directly costs 2^T operations; the forward recursion costs T times the square of the number of states, by reusing the work done at the previous position.

Tasks:

e) Define the forward variable alpha(t, s) in words, as a joint probability of something observed and something hidden. Be precise about which observations it covers

f) State the initialisation, the recursion and the termination step, and implement them. Report P(observations) for the supplied 30-roll sequence

g) Verify the implementation on a short sequence of length 5 by also summing over all 2^5 state paths explicitly. The two values must agree to machine precision

h) Count the arithmetic operations performed by each method for T = 5, T = 20 and T = 300, and state the growth rate of each in terms of T and the number of states

Problem Statement 3: Numerical Underflow

The forward variables are probabilities of ever-longer sequences, so they shrink geometrically. On any real sequence length they reach zero in double precision, and every subsequent result becomes meaningless.

Tasks:

i) Run the forward algorithm on generated sequences of length 50, 100, 200, 300, 400, 500 and 800, recording the smallest forward value and P(observations) in each case. Report the length at which the result first underflows to exactly zero

j) Implement the log-space remedy: carry log alpha throughout, replacing each sum of probabilities with a numerically safe log-sum-exp. Confirm it agrees with the direct version wherever the direct version still works

k) Implement the scaling remedy instead: rescale the forward vector to sum to 1 at each position and accumulate the logarithms of the scale factors. Confirm it gives the same log P(observations)

l) State one advantage of each remedy over the other, and say which one the backward pass of the next problem statement makes more convenient

Problem Statement 4: The Backward Pass, Filtering and Smoothing

The forward variable uses only the observations up to position t. Combining it with a backward variable that carries the observations after t gives the posterior over the state at t given the whole sequence.

Tasks:

m) Define the backward variable beta(t, s) in words and state its recursion, which runs from the end of the sequence toward the start. Implement it

n) Verify that the forward pass and the backward pass agree on P(observations), computed from either end

o) Combine them into the smoothed posterior, the probability that the die was loaded at each position given all 30 rolls. Confirm the two state probabilities sum to 1 at every position

p) Distinguish filtering from smoothing: filtering uses the observations up to t, smoothing uses all of them. Compute both for the supplied sequence and plot or tabulate them together

q) Identify the position at which the two curves differ most, and explain what the smoother knows at that position that the filter does not

Problem Statement 5: The Viterbi Algorithm

Viterbi finds the single most likely sequence of hidden states. The recursion is the forward recursion with the sum replaced by a maximum, together with a record of which predecessor achieved each maximum.

Tasks:

r) State the Viterbi recursion beside the forward recursion and identify the single operation that differs

s) Implement it in log space, with a back-pointer table, and recover the path by tracing the back-pointers from the end. Report the path for the supplied 30-roll sequence and the log probability of that path

t) Explain why the back-pointer table is necessary, and what goes wrong if the path is instead reconstructed by taking the highest-scoring state at each position after the fact

u) Generate 300 rolls together with their true hidden path, run Viterbi, and report the proportion of positions recovered correctly. Repeat over ten generated sequences and report the mean

Problem Statement 6: Viterbi Against Posterior Decoding

Posterior decoding takes the most likely state at each position independently. It answers a different question from Viterbi, and the two answers differ.

Tasks:

v) Produce both decodings for the supplied 30-roll sequence and report the number of positions at which they disagree

w) Repeat over ten generated sequences of length 300. Report, for each decoding, the proportion of positions matching the true hidden path, and the number of positions on which the two decodings disagree

x) Posterior decoding frequently matches more individual positions than Viterbi, yet Viterbi is the standard choice in most applications. Explain what each method optimises, and describe a transition matrix under which posterior decoding would return a state sequence of probability exactly zero

DELIVERABLES

A single Python file containing all six problem statements, together with its printed output and the written answers placed as comments beside the task requiring them.

REFERENCES

1. L. R. Rabiner, A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition, Proceedings of the IEEE, 1989.

2. R. Durbin, S. Eddy, A. Krogh and G. Mitchison, Biological Sequence Analysis, Cambridge University Press, 1998, Chapter 3.

3. D. Koller and N. Friedman, Probabilistic Graphical Models: Principles and Techniques, MIT Press, Chapter 6.

4. K. Murphy, Machine Learning: A Probabilistic Perspective, Chapter 17.